

PARTE 1: O BACKEND (Para o Arquiteto/Equipe NestJS)

1. A Base Principal (O Alicerce):

- **Repositório:** [brocoders/nestjs-boilerplate](#)
- **O que fazer:** Vocês vão dar um `git clone` neste repositório e ele será o projeto oficial do backend de vocês. Ele vai cuidar do banco de dados (lembrem de mudar para PostGIS), do Docker, do login comum e do painel web.

2. O Complemento (Apenas o que copiar do PowerSync):

- **Repositório de Consulta:**
[powersync-ja/powersync-nodejs-backend-todolist-demo](#)
- **O que fazer:** Vocês **NÃO** vão clonar este projeto inteiro. O arquiteto vai apenas abrir os arquivos dele no GitHub para **COPIAR DUAS COISAS** e colar dentro do projeto do Brocoders:
 1. **A Rota de Token (Auth):** O Flutter precisa de um Token especial para baixar o banco offline. O arquiteto vai olhar como o PowerSync gera esse JWT e criar uma rota no NestJS (ex: `/api/powersync/token`).
 2. **A Rota de Sincronização (Upload):** O arquiteto vai copiar a lógica que recebe os dados do celular (quando a internet volta) e salva no PostgreSQL.
- **Resumo do Backend:** É o projeto do Brocoders, mas "injetando" duas rotas copiadas do exemplo do PowerSync.

PARTE 2: O FRONTEND (Para a Equipe Flutter)

Aqui a estratégia é o inverso. Fazer a conexão offline do zero é muito difícil, então vamos usar o exemplo do PowerSync como base e colocar as ferramentas de mapa por cima.

1. A Base Principal (O Aplicativo):

- **Repositório:** O monorepo [powersync-ja/powersync.dart](#)
- **O que fazer:** O pessoal do front vai baixar esse repositório inteiro, mas vai apagar quase tudo. Eles vão **pegar APENAS a pasta** chamada `/demos/custom_backend_todolist`.
- **Por que isso?** Essa pasta é um aplicativo Flutter completinho e isolado. Ele já tem o banco SQLite configurado e já sabe como enviar dados para um servidor (o NestJS de vocês).

2. Os Complementos (Como transformar esse "To-Do List" no App Ambiental):

- A equipe de Flutter vai abrir esse projeto base e instalar pacotes extras (Bibliotecas do Flutter) para atingirmos o System Design:
 - Instalar o pacote `flutter_map`: Para desenhar o polígono da fazenda na tela.
 - Instalar o pacote `flutter_map_tile_caching`: Para o mapa carregar a imagem de satélite mesmo sem 3G/4G.

- Instalar o pacote **geolocator**: Para pegar a coordenada exata de onde o técnico está pisando.
- Instalar o pacote **camera**: Para bater a foto.
- **O que a equipe de Front faz**: Eles apagam as telas de "Lista de Tarefas" (To-Do) que vêm no exemplo, e desenham as telas do PRAD (Formulário, Mapa e Foto) usando o SQLite que já está lá pronto e conectado.

RESUMO DA ÓPERA (Como as pontas se encontram)

1. O **Frontend** (usando a base **custom_backend_todolist** do Flutter) tenta baixar as missões da fazenda.
2. Ele bate lá no seu **Backend** (usando o **nestjs-boilerplate**).
3. O seu Backend olha a rota que o seu arquiteto copiou, valida quem é o técnico e devolve um Token.
4. O celular do técnico baixa o mapa e o polígono. Ele vai pro meio da floresta.
5. No mato, ele tira a foto (**camera**), pega a coordenada (**geolocator**) e salva no SQLite local.
6. Quando ele volta pra cidade, o aplicativo manda o formulário preenchido de volta para o NestJS.